

CSE 331: Computer Security

Fundamentals

Spring 2026

Kernel & Memory Management

Amir Rahmati
Xinrui Yu



Stony Brook Institute
at Anhui University

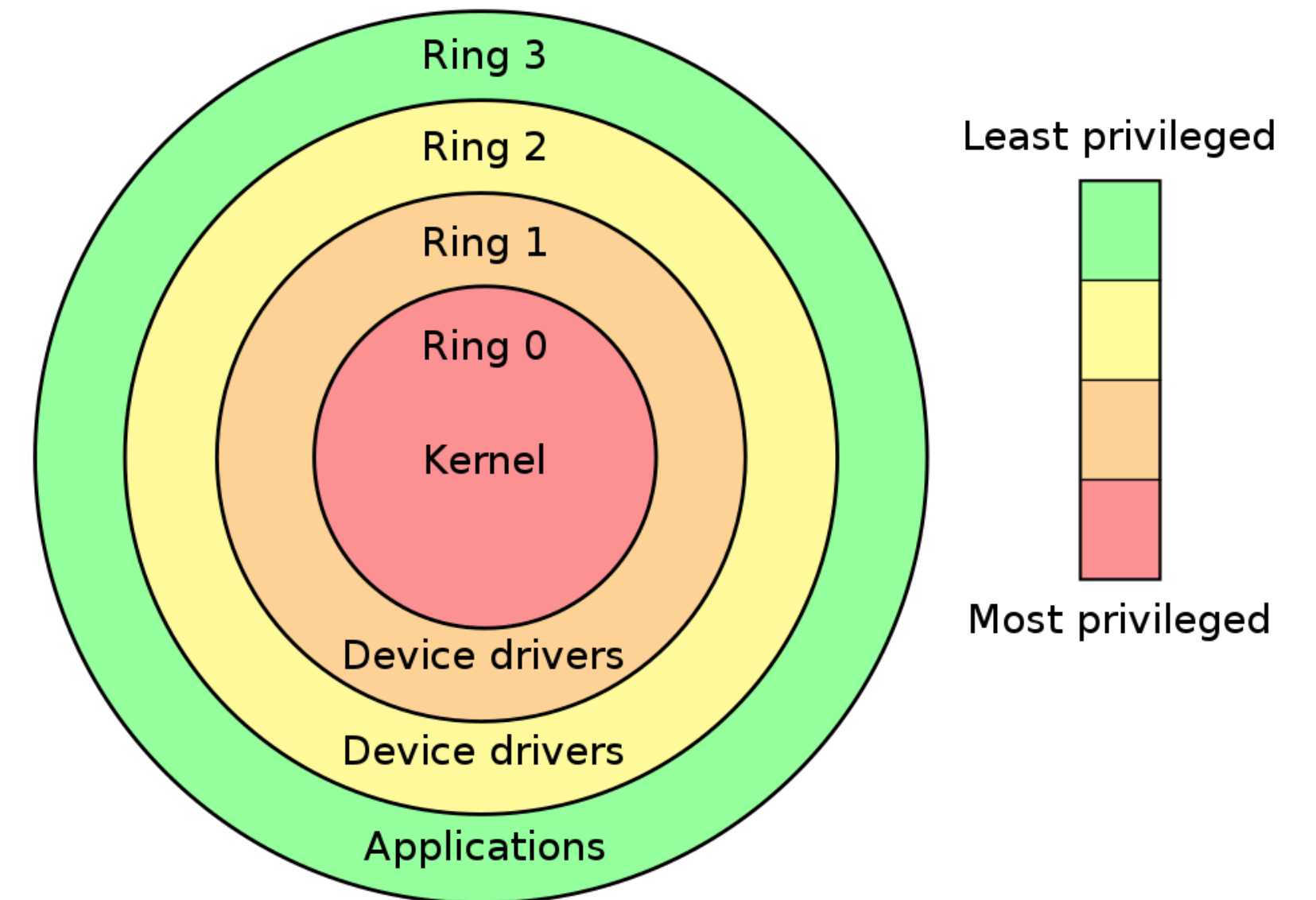
安徽大学纽约石溪学院

What does Kernel do?

- The kernel is responsible for handling the processes' interactions with each other and with external resources.
- Examples of kernel-only resources:
 - `hlt` instruction: shuts CPU computation.
 - `in` and `out` instructions for interacting with hardware peripherals.
 - Special registers
 - `cr3` (Control Register 3), which controls the page table used to translate virtual addresses to physical addresses. Accessed using `MOV`. https://en.wikipedia.org/wiki/Control_register
 - `MSR_LSTAR` (Model-Specific Register, Long Syscall Target Address Register), which defines where the `syscall` instruction jumps to. Accessed using `wrmsr` and `rdmsr`
<https://0xax.gitbooks.io/linux-insides/content/SysCall/linux-syscall-1.html>
- How does the computer know whether your code can access these resources?

Privilege levels

- The CPU tracks a privilege level that controls access to resources. This is generally split into "rings" of access:
 - **Ring 3:** Userspace, where we have been operating until now. Very restricted.
 - **Ring 2:** Generally unused.
 - **Ring 1:** Generally unused.
 - **Ring 0:** The Kernel. Unrestricted, supervisor mode.
- Similar to an OS tracking your user ID, the CPU tracks the current privilege level.



One ring to rule them all

- Supervisor Mode's privileges started causing issues with the rise of Virtual Machines. A VM's "guest" kernel shouldn't be able to have unlimited access to the host's physical hardware.
- **Early 2000s solution:** force the VM kernel into Ring 1.
 - **Drawback:** have to use costly and complex emulation methods to simulate some Ring 0 actions of the guest OS.
- **Modern solution:** Ring -1, Hypervisor Mode. Able to intercept sensitive Ring 0 actions done by guests and handle them in the host OS.

Different types of OS models



- In a **monolithic kernel** there is a single, unified kernel binary that handles all OS-level tasks. Drivers are libraries loaded into this binary.
 - examples: Linux, FreeBSD
 - In a **microkernel**, there is a tiny "core" binary that provides inter-process communication and barebone interactions with the hardware. Drivers are normal-ish userspace programs with slightly special privileges.
 - examples: Minux, seL4
 - In a **hybrid kernel**, microkernel features are combined with a monolithic component.
 - examples: Windows (NT), MacOS
- Any vulnerability in device driver is a kernel vulnerability!**

Switching between rings

1. At bootup, in Ring 0, the kernel sets `MSR_LSTAR` to point to the syscall handler routine (`sysenter`).
2. When a userspace (Ring 3) process wants to interact with the kernel, it can call `syscall`.
 - A. Privilege level switches to Ring 0.
 - B. Control flow jumps to value of `MSR_LSTAR`. (Set the PC to the value in `MSR_LSTAR`)
 - C. Return address saved to `rcx`.
 - D. A bunch of other housekeeping: <https://www.felixcloutier.com/x86/syscall>
3. When the kernel is ready to return to userspace, it calls the appropriate return instruction (i.e., `sysret` for `syscall`).
 - A. Privilege level switches to Ring 3.
 - B. Control flow jumps to `rcx`. (Set the PC to the value in `RCX`)

Kernel-Userspace Relationship

- `cat /proc/self/maps`
- Userspace processes have their virtual memory at low addresses.
- The kernel has its own virtual memory space, but in high addresses!
- System calls do NOT switch the virtual memory mapping, but kernel memory is only accessible from Ring 0!

```
56124bc0e000-56124bc10000 r--p 00000000 fd:01 7864471 /usr/bin/cat
56124bc10000-56124bc15000 r-xp 00002000 fd:01 7864471 /usr/bin/cat
56124bc15000-56124bc18000 r--p 00007000 fd:01 7864471 /usr/bin/cat
56124bc18000-56124bc19000 r--p 00009000 fd:01 7864471 /usr/bin/cat
56124bc19000-56124bc1a000 rw-p 0000a000 fd:01 7864471 /usr/bin/cat
56124c07f000-56124c0a0000 rw-p 00000000 00:00 0 [heap]
7fae08b4a000-7fae08b6c000 rw-p 00000000 00:00 0
7fae08b6c000-7fae090dc000 r--p 00000000 fd:01 7864602 /usr/lib/locale/locale-archive
7fae090dc000-7fae09101000 r--p 00000000 fd:01 7866483 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fae09101000-7fae09279000 r-xp 00025000 fd:01 7866483 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fae09279000-7fae092c3000 r--p 0019d000 fd:01 7866483 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fae092c3000-7fae092c4000 ---p 001e7000 fd:01 7866483 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fae092c4000-7fae092c7000 r--p 001e7000 fd:01 7866483 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fae092c7000-7fae092ca000 rw-p 001ea000 fd:01 7866483 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fae092ca000-7fae092d0000 rw-p 00000000 00:00 0
7fae092ee000-7fae092ef000 r--p 00000000 fd:01 7866478 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fae092ef000-7fae09312000 r-xp 00001000 fd:01 7866478 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fae09312000-7fae0931a000 r--p 00024000 fd:01 7866478 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fae0931b000-7fae0931c000 r--p 0002c000 fd:01 7866478 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fae0931c000-7fae0931d000 rw-p 0002d000 fd:01 7866478 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fae0931d000-7fae0931e000 rw-p 00000000 00:00 0
7ffec1e8c000-7ffec1ead000 rw-p 00000000 00:00 0 [stack]
7ffec1fa5000-7ffec1fa8000 r--p 00000000 00:00 0 [vvar]
7ffec1fa8000-7ffec1fa9000 r-xp 00000000 00:00 0 [vdso]
fffffffff600000-fffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

Kernel Vulnerabilities

- Code in the kernel is just code! Most of the same vulnerability concepts apply!
- Kernel exploits can come from a few directions:
 - **From the network:** remotely-triggered exploits (packets of death, etc). Rare!
 - **From userspace:** vulnerabilities in syscall and ioctl handlers (i.e., launched from inside a sandbox!)
 - **From devices:** launch kernel exploits from attached devices such as USB hardware (<https://www.pjrc.com/teensy/>)
- And can achieve a number of things:
 - **Act on userspace:** privilege escalation, rootkits
 - **Get more access** to attack other parts of the system (i.e., trusted execution environments)

Monolithic kernel

- Single, unified binary. Other stuff are loaded in as **modules**.
- A kernel module is a library that loads into the kernel.
- Similar to a userspace library (i.e., `/lib/x86_64-linux-gnu/libc.so.6`):
 - The module is an executable file (.ko extension instead of .so).
 - The module is loaded into the address space of the kernel.
 - Code in the module runs with the same privileges as the kernel.
- Kernel modules are used to implement:
 - device drivers (graphics cards, etc)
 - filesystems
 - networking functionality
 - ...

Module Interaction: System Calls

- Historically, kernel modules could add system call entries through a bit of effort by modifying the kernel's system call table.
 - Often used by rootkits
 - This is explicitly unsupported in modern kernels.

Module Interaction: Interrupts

- Theoretically, a module could register an interrupt handler by using the LIDT and LGDT instructions and be triggered by an `int 42` instruction.
- Useful one-byte interrupt instructions to hook:
 - `int3 (0xcc)` : normally causes a SIGTRAP, but can be hooked!
 - `int1 (0xf1)` : normally used for hardware debugging, but can be hooked!
- A module can also hook the Invalid Opcode Exception interrupt!
 - can be used to implement custom instructions in software
- Not really used – modern Linux kernels do not encourage or officially support arbitrary hooking.

Module Interaction: Files

- The most common way of interacting with modules is via file!
 - `/dev`: mostly traditional devices (i.e., `/dev/dsp` for audio)
 - `/proc`: started out in System V Unix as information about running processes. Linux expanded it into in a disastrous mess of kernel interfaces.
 - `/sys`: non-process information interface with the kernel.
- A module can register a file in one of the above locations.
- Userspace code can `open()` that file to interact with the module!

File read() and write()

- One interaction mode is to register handler read() and write() for your module's exposed file.

- From kernel space:

```
static ssize_t device_read(struct file *filp, char *buffer, size_t length, loff_t
*offset)
static ssize_t device_write(struct file *filp, const char *buf, size_t len, loff_t
*off)
```

- From user space:

```
fd = open( "/dev/labtainer", 0)
read(fd, buffer, 128);
```

- Useful for modules that deal with streams (i.e., a stream of audio or video data).

File ioctl()

- Input/Output Control provides a much more flexible interface.
- From kernel space:

```
static long device_ioctl(struct file *filp, unsigned int ioctl_num, unsigned long  
ioctl_param)
```

- From user space:

```
int fd = open( “/dev/labtainer”, 0);  
ioctl(fd, COMMAND_CODE, &custom_data_structure);
```

- Useful for setting and querying non-stream data (i.e., webcam resolution settings as opposed to webcam video stream).

Driver Interaction: Inside the Kernel

- The kernel can do anything, and kernel modules in a monolithic kernel are the kernel.
- But typically, the kernel:
 1. reads data from userspace (using `copy_from_user`)
 2. "does stuff" (open files, read files, interact with hardware, etc)
 3. writes data to userspace (using `copy_to_user`)
 4. returns to userspace

Interacting with a Module

- **Loading:** `init_module` system call. usually done through the `insmod` utility.

```
# insmod mymodule.ko
```

- **Listing:** Loaded kernel modules can be seen using:

```
# lsmod
```

- **Removal:** `delete_module` system call, usually done through the `rmmod` utility:

```
# rmmod mymodule
```

How can vulnerabilities happen?

- Each module has two important functions:
 - `copy_to_user(userspace_address, kernel_address, length);`
 - `copy_from_user(kernel_address, userspace_address, length);`
- **Must use with Caution – Possibility of memory corruption!**
- Kernel memory must be kept uncorrupted! Corruption can:
 - crash the system
 - brick the system
 - escalate process privileges
 - interfere with other processes
- All user data should be carefully handled and only accessed with `copy_to_user` and `copy_from_user`.

Memory Corruption

- **Missing pointer checks:** Not validating the value of user-provided pointers or index variables to ensure that they point to user-space memory only.
- **Missing permission checks:** Performing a privileged operation without checking whether the calling process has the privilege to do so.
- **Buffer overflow:** Incorrectly checking the upper/lower bound when accessing a buffer, allocating a smaller buffer than it is supposed to, using unsafe string manipulation functions, or defining local variables which are too large for the kernel stack.
- **Integer overflow:** Performing an integer operation incorrectly, resulting in an integer overflow, underflow, or sign error.
- **Uninitialized data:** Copying the contents of a kernel buffer to user space without zeroing unused fields, thus leaking potentially sensitive information to user processes

Kernel Vulnerabilities Happen

- Kernel code is just code!
- Memory corruptions, allocator misuse, etc, all happen in the kernel!
- What can you do with this?

Privilege Escalation

- The kernel tracks user the privileges (and other data) of every running process.

```
struct task_struct {
    struct thread_info    thread_info;

    /* -1 unrunnable, 0 runnable, >0 stopped: */
    volatile long         state;

    void                  *stack;
    atomic_t              usage;
    // ...
    int                   prio;
    int                   static_prio;
    int                   normal_prio;
    unsigned int          rt_priority;

    struct sched_info     sched_info;

    struct list_head      tasks;

    pid_t                 pid;
    pid_t                 tgid;

    /* Process credentials: */

    /* Objective and real subjective task credentials (COW): */
    const struct cred __rcu *real_cred;

    /* Effective (overridable) subjective task credentials (COW): */
    const struct cred __rcu *cred;
    // ...
};
```

```
struct cred {
    atomic_t              usage;
    kuid_t                uid;      /* real UID of the task */
    kgid_t                gid;      /* real GID of the task */
    kuid_t                suid;     /* saved UID of the task */
    kgid_t                sgid;     /* saved GID of the task */
    kuid_t                uid;      /* effective UID of the task */
    kgid_t                egid;     /* effective GID of the task */
    kuid_t                fsuid;     /* UID for VFS ops */
    kgid_t                fsgid;     /* GID for VFS ops */
    unsigned              securebits; /* SUID-less security management */
    kernel_cap_t           cap_inheritable; /* caps our children can inherit */
    kernel_cap_t           cap_permitted;   /* caps we're permitted */
    kernel_cap_t           cap_effective;    /* caps we can actually use */
    kernel_cap_t           cap_bset;        /* capability bounding set */
    kernel_cap_t           cap_ambient;     /* Ambient capability set */
    // ...
};
```


How do we set these?

- The credentials are supposed to be immutable (i.e., they can be cached elsewhere, and shouldn't be updated in place). Instead, they can be replaced:

```
commit_creds(struct cred *)
```

- The cred struct seems a bit complex, but the kernel can make us a fresh one!

```
struct cred * prepare_kernel_cred(struct task_struct *reference_task_struct)
```

- If we pass NULL to the reference struct, it'll give us a cred struct with root access and full privileges!
- We have to run: `commit_creds(prepare_kernel_cred(0));`

Complications

- How do we know where `commit_creds` and `prepare_kernel_cred` are in memory?
 - Older kernels (or newer kernels when kASLR is disabled) are mapped at predictable locations.
 - `/proc/kallsym` is an interface for the kernel to give root these addresses.
 - If enabled, `gdb` support is your friend.
 - Otherwise, it's the exact same problem as userspace ASLR!

Kernel Race Conditions

- Kernel modules are not userspace programs.
 - They are always prone to multi-threading
 - They could disappear or swap resources mid-execution
- Race conditions are huge problems plaguing kernels!

Memory Management

- Each Linux process has a virtual memory space. It contains:
 - the binary
 - the libraries
 - the "heap" (for dynamically allocated memory)
 - the "stack" (for function local variables)
 - any memory specifically mapped by the program
 - some helper regions
 - **kernel code** in the "upper half" of memory (above 0x8000000000000000 on 64-bit architectures), inaccessible to the process
- **Virtual memory** is dedicated to your process.
- **Physical memory** is shared among the whole system.
- You can see this whole space by looking at `/proc/self/maps`

Physical Memory

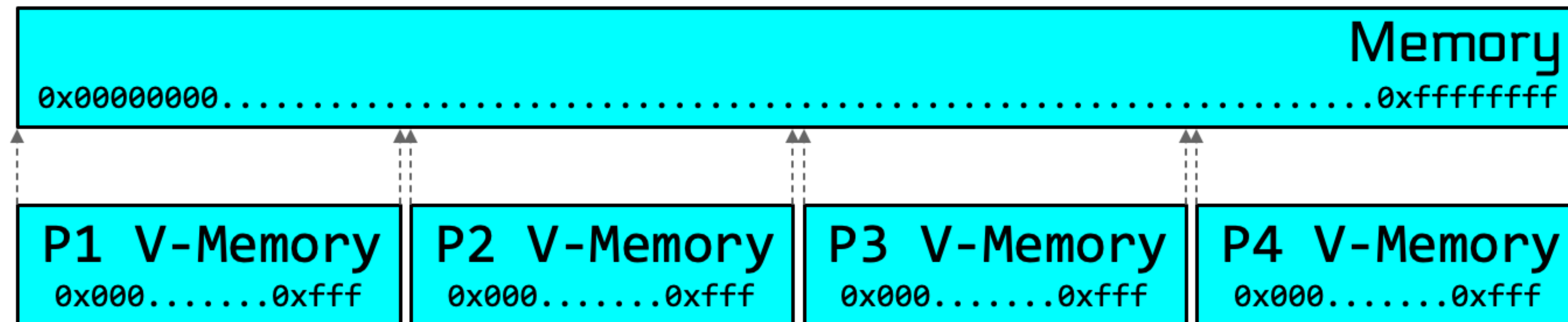
- Physical memory is the RAM inside your computer:



- But there can only be one physical 0x400100... How would we load multiple programs into memory?
- One solution: position independent code (like libraries!). But what about isolation?

Virtual Memory

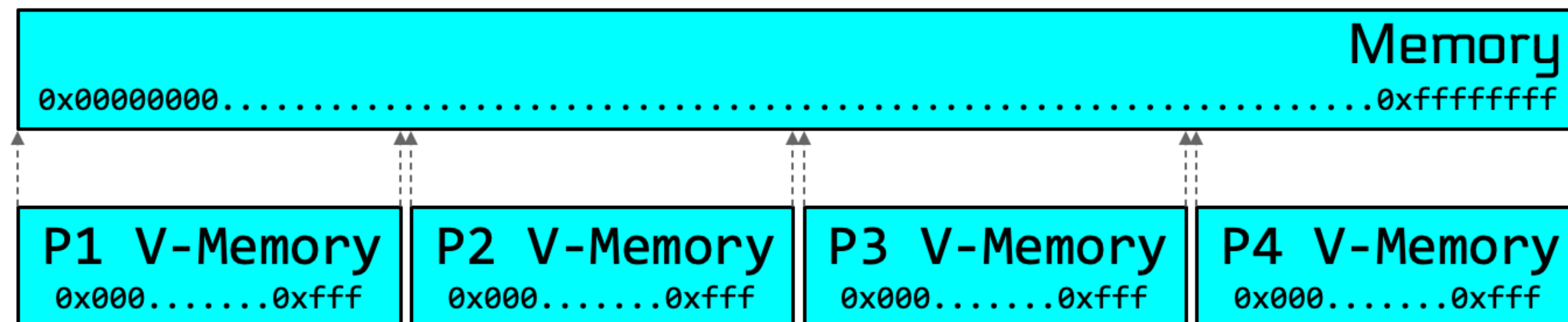
- Virtual Memory provides different views into physical memory!



- The OS kernel (in collaboration with the CPU architecture) maintains mappings between a process' virtual memory addresses and the actual addresses they correspond to in physical memory.

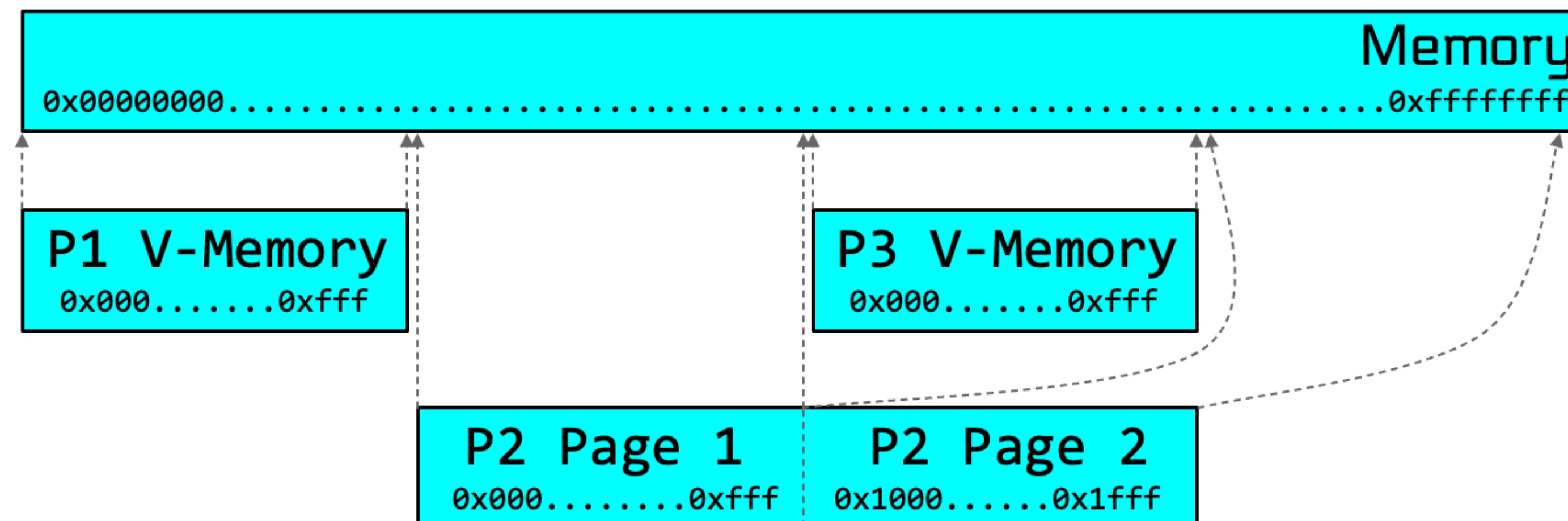
How to Map Virtual and Physical Memory?

- Strawman solution:
Each process gets ... 4kb of memory! (who would ever need more?)
Each process' entire virtual memory gets mapped at some place in physical memory.
- Pro: Easy to track the mappings!
- Con: What if P2 needs to allocate more memory



Solution: non-contiguous physical memory

- Track the physical base address of pages of virtual addresses.
- The physical position of contiguous virtual pages can be non-contiguous.



The Page Table

- Diving back into history, we start with the Page Table full of Page Table Entries.

Page Table	
000	0xaa201000
001	0x421e0000
002	0x5c644000
003	0xf1f0b000
004	0x1cc00000
...	
511	0xfa115000

- Each page table holds 512 entries, mapping up to 2MB of memory.
- What about non-contiguous virtual memory?
What if we need more than 2MB of memory?

Supporting Non-Contiguous Virtual Memory

- Introducing: the multi-level paging structure.
- A Page Directory contains 512 Page Directory Entries, each mapping a page table, each holding 512 entries, each referring a page of 0x1000 bytes, for a total of 1GB of addressable virtual memory.
- Overhead:
0x1000 bytes for the PD
0x1000 bytes (per PT) for each 2MB of memory.
- Overhead Reduction:
By setting a special flag, a PDE can refer to a 2MB region of physical memory (instead of a PT).

Page Directory	
000	PT A
001	NONE
002	PT B
003	NONE
004	NONE
...	
511	PT C

Page Table A	
000	0xaa201000
001	0x421e0000
002	0x5c644000
003	0xf1f0b000
004	0x1cc00000
...	
511	0xfa115000

Page Table B	
000	0xffe01000
001	0xcc1e0000
002	0x13d44000
003	0xfcfeb000
004	0x462e0000
...	
511	0xbb315000

What if we need more than 1GB of RAM?

- A Page Directory Page Table contains 512 Page Directory Pointers, addressing a total of 512GB of RAM!
- Overhead:
0x1000 for the PDPT.
0x1000 * 512 for the PDs.
- Overhead Reduction:
By setting a special flag, a PDP can refer to a 1GB region of physical memory (instead of a PD).

PDPT	
000	NONE
001	NONE
002	PDP A
003	NONE
004	PDP B
...	
511	NONE

Page Directory A	
000	PT A
001	NONE
002	PT B
003	NONE
004	NONE
...	
511	PT C

Page Table A	
000	0xaa201000
001	0x421e0000
002	0x5c644000
003	0xf1f0b000
004	0x1cc00000
...	
511	0xfa115000

Page Directory B	
000	PT D
001	PT E
002	NONE
003	NONE
004	NONE
...	
511	PT F

Page Table B	
000	0xffe01000
001	0xcc1e0000
002	0x13d44000
003	0xfcfeb000
004	0x462e0000
...	
511	0xbb315000

512GB is getting small....

- Modern systems have 4 levels of paging.
- A Page Map Level 4 maps 512 Page Directory Page Tables, addressing a theoretical total of 256 TB of RAM.

PML4	
000	NONE
001	PDPT A
002	NONE
003	NONE
004	PDPT B
...	
511	NONE

PDPT A	
000	NONE
001	NONE
002	PDP A
003	NONE
004	PDP B
...	
511	NONE

Page Directory A	
000	PT A
001	NONE
002	PT B
003	NONE
004	NONE
...	
511	PT C

Page Table A	
000	0xaa201000
001	0x421e0000
002	0x5c644000
003	0xf1f0b000
004	0x1cc00000
...	
511	0xfa115000

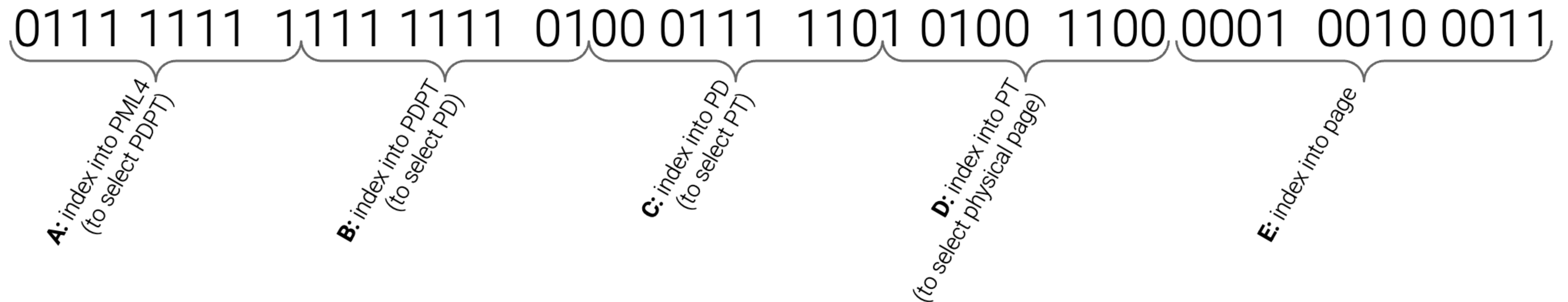
PDPT B	
000	NONE
001	NONE
002	NONE
003	NONE
004	PDP C
...	
511	NONE

Page Directory B	
000	PT D
001	PT E
002	NONE
003	NONE
004	NONE
...	
511	PT F

Page Table B	
000	0xffe01000
001	0xcc1e0000
002	0x13d44000
003	0xfcfeb000
004	0x462e0000
...	
511	0xbb315000

A Virtual Address

- Consider 0x7fff47d4c123. In binary:



- This is "only" 48 bits of addressable memory.

`mov rax, [rbx]`

becomes

`rax = *(long *) (PML4[A][B][C][D])[E]`

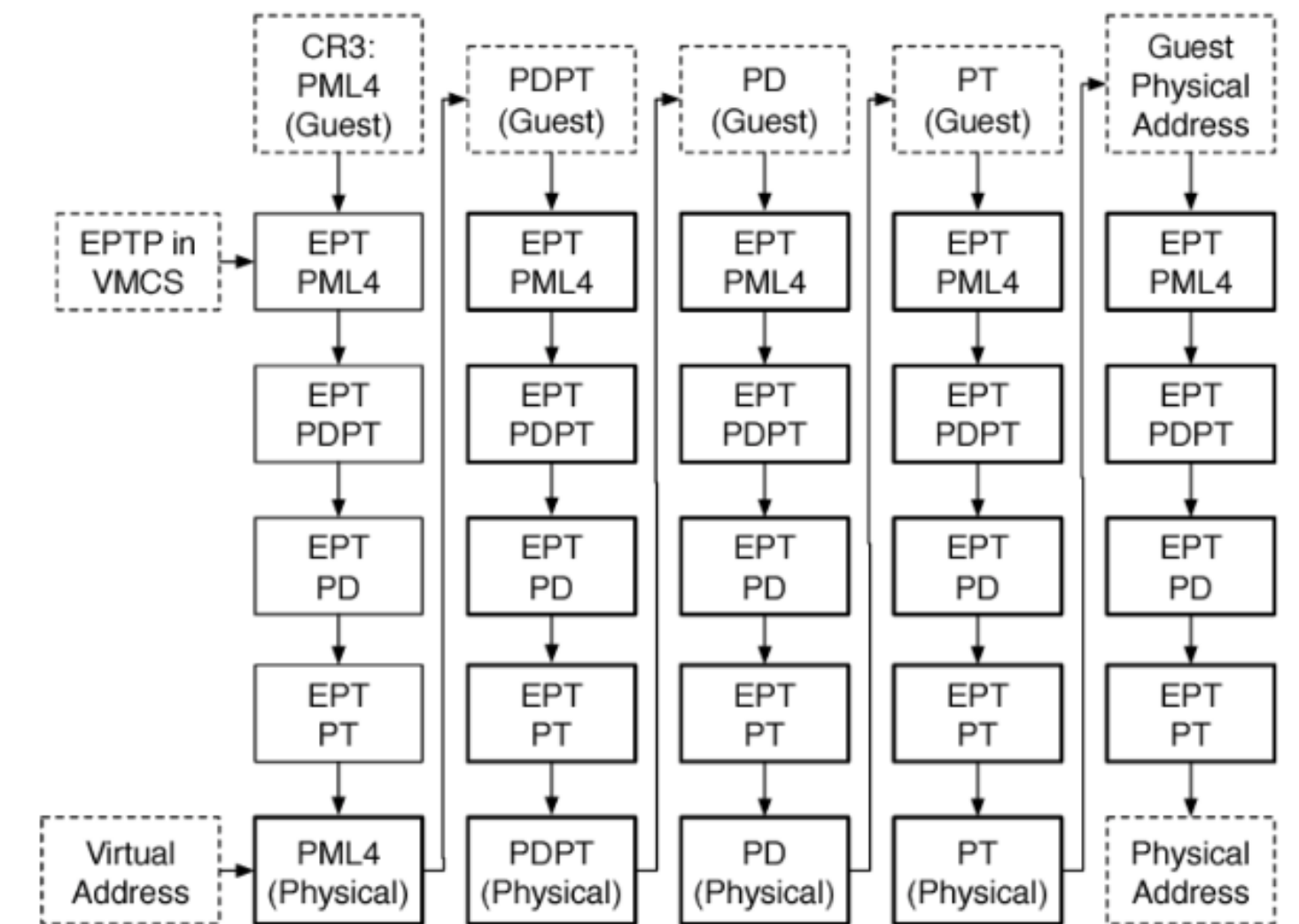
Process Isolation

- Each process has its own PML4! How to find it?
- The CR3 register holds the physical location of the PML4.
`mov cr3, 0xaabbccdd`
- ONLY accessible in ring 0!
- Other Control Registers exist, setting processor options (including whether we are in 32-bit or 64-bit mode) and lots of other craziness.
- For the interested: https://wiki.osdev.org/CPU_Registers_x86

Virtual Machine Isolation?

- How do we isolate multiple virtual machines, the kernels of whom expect to have full access to physical memory?
- The Extended Page Table.
ANOTHER ADDRESS TRANSLATION TABLE!
- Every VM thinks it has access to physical memory.
- In reality, each "physical" memory access is translated through the EPT analogous to a virtual memory translation.

EPT PML4 → EPT PDPT → EPT PD → EPT PT



The Two-dimensional lookup process...
<https://rayanfam.com/topics/hypervisor-from-scratch-part-4/>

Hardware support for lookups

- It would be too slow for the kernel to do all of these lookups in software...
- Introducing: the Memory Management Unit.
The MMU does the lookups in hardware. Blazing fast!
- Resolved addresses are cached in the Translation Lookaside Buffer.

Other Architectures

- So far, we focused on x86. But other architectures are analogous!
- ARM. (Simplified view. Full details: <https://developer.arm.com/documentation/100940/0101>)
CR3 equivalent: **TTBR0** (for userspace) and **TTBR1** (for kernel addresses).
Translation tables are referred to as Level 0, Level 1, Level 2, Level 3.
- Linux: generic terminology.
 - PML4 = PGD (Page Global Directory)
 - PDPT = PUD (Page Upper Directory)
 - PD = PMD (Page Mid-Level Directory)
 - PT = PT (Page Table)
- Linux requires a hardware MMU (although certain forks do not).

Many Mitigations Available

The kernel has many mitigations. Some are familiar:

Stack canaries protect the stack

kASLR bases the kernel at a random location *at boot*.

Heap/stack regions are not executable by default.

The kernel's philosophy: <https://www.kernel.org/doc/Documentation/security/self-protection.txt>

... with the expected bypasses.

Of course, you know how to bypass these:

Stack canaries: leak the canary!

kASLR: leak the kernel base address!

Heap/stack regions NX: ROP!

Solutions? One crazy idea in kASLR: shuffle functions around!

Function Granular ASLR.

Upcoming mitigation to prevent kASLR bypass via trivial address disclosure.

More info: <https://lwn.net/Articles/824307>

Kernel-Specific Mitigation: Supervisor Memory Protection

Many exploits trick the kernel from accessing or executing user-space memory.

SMEP.

Prevents kernel-space code from **E**xecuting userspace memory *at all ever*.

SMAP.

Prevents kernel-space from even **A**ccessing userspace memory *unless the AC flag in the RFLAGS register is set*. Two ring0 instructions, **stac** and **clac**, manage this bit.

Why separate these? There *are* cases where the kernel needs access to userspace memory (e.g., to access the **path** argument to **open()**).

copy_from_user sets the AC flag to access userspace.

More details: https://wiki.osdev.org/Supervisor_Memory_Protection